SteadyDet-N: A 4.6-GFLOP Person Detector That Leads Every Compact Baseline Under UAV Vibration

Nhan Le Thanh Huynh[1] and [Advisor Name][1]

Ho Chi Minh City University of Technology and Education, Ho Chi Minh City, Vietnam
[your.email@hcmute.edu.vn]

Abstract. We present SteadyDet-N, the nano variant of our SteadyDet family: a 1.17 M-parameter, 4.6-GFLOP anchor-free person detector for multirotor UAVs, and show that it leads every compact baseline we measured on the metric its deployment depends on, while losing to all of them on the benchmark. That metric is accuracy under airframe vibration. Multirotors vibrate whenever the rotors turn, and under a single evaluation protocol with seeded corruption and error bars every compact detector we tested retains at most a quarter of its clean average precision and is effectively blind at the moderate severity of ordinary forward flight. The detector does not survive this by architecture alone; untreated, it is the worst network in our table. What closes the gap is deliberately plain. We replace half of each training batch with a copy corrupted by a physically motivated model of motion blur, sensor noise and exposure oscillation, and compute the ordinary detection loss on it. The corruption preserves object coordinates so the labels stay valid, and it exists only during training, so the deployed network is unchanged. Retention rises from $16.8 \pm 0.7\%$ to $36.2 \pm 0.7\%$ and accuracy under moderate vibration by $4.2\times$, at a cost of 9.7% clean AP, enough to overtake all four YOLO nano variants under vibration and a transformer of $28\times$ the size at the severity of ordinary flight. Neither half suffices alone: the same regime applied to YOLO11-n changes nothing, from scratch or from pretrained weights, and we rule out pretraining, object scale and remaining headroom as explanations. Two further controls make the result interpretable. Finetuning for the same 120 epochs without corruption leaves retention unchanged ($16.9 \pm 0.7\%$), and a consistency formulation we expected to be stronger reaches only $30.0 \pm 0.6\%$ at $1.8\times$ the training cost. We report both negative results in full, since together they identify what accuracy under vibration actually depends on.

Keywords: Tiny object detection · Robustness · Data augmentation · UAV · Edge deployment.

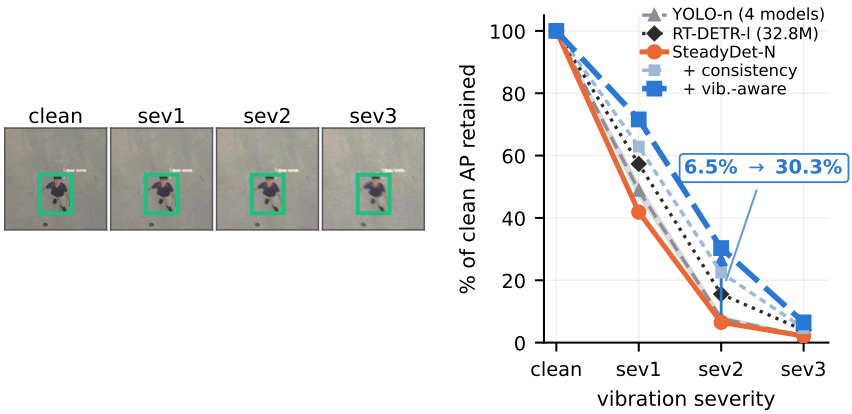


Fig. 1. Compact aerial detectors go blind when the airframe vibrates. A flight frame under each VisDrone-Shake severity (sev1 and sev2 are ordinary flight, not a fault), and the fraction of clean AP that survives on VisDrone-person, over three seeds. Every published detector collapses at the same rate: the failure belongs to the regime, not to one network.

1 Introduction

A detector that scores well on an aerial benchmark can still be unusable on an actual aircraft, for a physical reason the literature largely ignores: a multirotor vibrates. Rotor imbalance, motor harmonics and frame resonance transmit high-frequency motion to the camera, and because the sensor integrates over a finite exposure, that motion appears as directional blur, elevated noise and frame-to-frame brightness oscillation. Unlike weather or occlusion this corruption is not occasional; it is present whenever the rotors are spinning.

We began by measuring how much it matters, on our own detector first. SteadyDet-N is an anchor-free network of 1.17 M parameters and 4.6 GFLOPs built for tiny-person detection from a UAV, and under a four-level vibration model it falls from 0.189 to 0.012 average precision at moderate severity, retaining $16.8 \pm 0.7\%$ of its clean accuracy. Fig. 1 shows the collapse is not ours alone: every published detector we measured falls at the same rate. A detector blind whenever the aircraft is airborne is not a detector, and on this axis the entire compact-detection literature is in the same position.

The obvious responses are unattractive on a small UAV. Isolation and gimbals cost mass, payload and power; deblurring adds a second network to a pipeline that must already fit a detector, a tracker and a controller into a 50 ms budget on a passively cooled board. Every one pays at run time, on the platform least able to afford it.

The response we test is deliberately blunt: put the vibration into the training distribution. Each batch is replaced, with probability one half, by a corrupted

copy drawn from a physically motivated model of rotor-induced degradation, and the ordinary detection loss is computed on whatever the network receives. Because blur, noise and brightness scaling do not move objects, the labels remain valid and nothing else changes. At inference the corruption does not exist and the exported network is identical to the baseline, so the robustness costs nothing where cost is felt.

Neither the network nor the regime suffices alone, and we show this rather than assert it. Untreated, SteadyDet-N is the least accurate row in our table under vibration; applied to YOLO11-n, the identical regime changes nothing at all, from COCO weights or from scratch. Only the combination moves, and it moves past all four YOLO nano variants while remaining the smallest and cheapest network compared. That null result is also what keeps the comparison fair: our treated detector is measured against a baseline that cannot be improved the same way, not against a handicapped one.

We also tried something more elaborate, a consistency formulation, and report that it is measurably worse at 1.8× the training cost. A control contradicting the authors' expectation is worth more to a reader than one confirming it.

Our contributions are: **(i)** *SteadyDet-N with its training regime*, which at 1.17 M parameters and 4.6 GFLOPs loses to every YOLO nano variant on clean VisDrone and leads all of them on accuracy under vibration, exceeding RT-DETR-l at 28× its size at the severity of ordinary forward flight (Section 4.3); **(ii)** *VisDrone-Shake*, a seeded four-level vibration protocol under which every detector we tested loses 74 to 83% of its average precision (Sections 3.2 and 4.3); **(iii)** *vibration-aware training*, raising retention from 16.8±0.7% to 36.2±0.7% at zero inference cost, with the control showing the 120 additional epochs contribute nothing on their own (16.9±0.7%) (Section 4.2); **(iv)** a controlled negative result, that the consistency formulation we expected to be better is measurably worse while costing more to train (Section 4.2); and **(v)** evidence that the architecture and the regime are each necessary, since the same regime applied to YOLO11-n produces no effect at all and three candidate explanations for that are tested and rejected (Section 4.2).

2 Related Work

Robustness to corruption. ImageNet-C [9] established the mean corruption protocol we adapt, later extended to detection [12]. Those benchmarks sample a broad corruption taxonomy suited to general vision; on a multirotor the distribution is concentrated instead, set by rotor dynamics and exposure time, so we build a domain-specific protocol rather than reuse a generic one. Motion deblurring [14] attacks the same degradation by restoring the image, adding a second network to the critical path; we leave the inference graph untouched.

Consistency regularisation. Agreement between perturbed views underpins semi-supervised learning [16,20,18] and self-supervised representation learning [2,8]. Our use differs in purpose: the data are fully labelled, so the term is not a

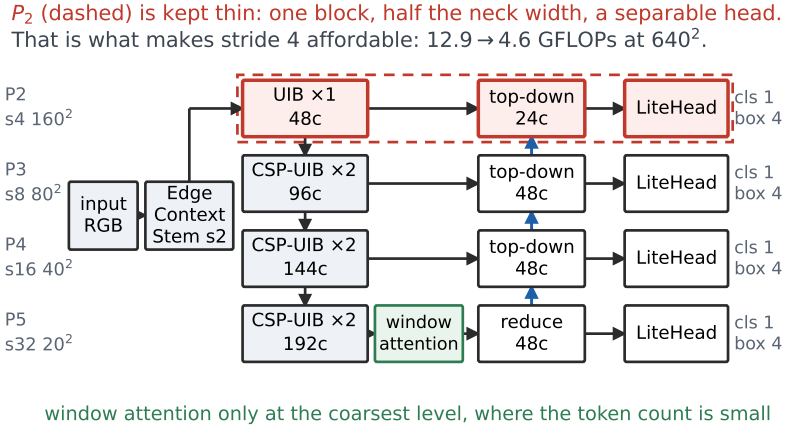


Fig. 2. SteadyDet-N; resolution decreases down the page. Compute is deliberately asymmetric: the stride-4 row P_2 (dashed), where most drone-scale persons are resolved, carries one block, a half-width neck branch and a separable head, while the two-block CSP stages and the attention sit where the maps are small. Blue arrows are the top-down upsample-and-fuse path. That is what keeps P_2 at 4.6 instead of 12.9 GFLOPs.

substitute for supervision but a directed statement about which nuisance to discard. The closest supervised analogue is consistency-based domain adaptation [6], where the perturbation models a domain gap rather than a sensing process.

Efficient aerial detection. VisDrone [26] is dominated by instances below 32×32 pixels, in the tiny-object regime of [24,4]. Efficient detectors here combine depthwise-separable backbones [15,17], cross-stage partial connections [22], feature pyramids [19] and anchor-free heads [7], with RemDet [3] targeting small-object downsampling and recent YOLO generations moving to NMS-free prediction [21]; IoU losses are unstable for tiny boxes, motivating NWD [23] blended with CIoU [25] alongside [11] and [5]. All of it optimises clean accuracy per unit of compute. We take a detector from this family and ask a different question: how much of that accuracy survives the aircraft it is meant to fly on.

3 Method

3.1 Detector Platform

SteadyDet-N is an anchor-free single-stage detector of 1.17 M parameters and 4.6 GFLOPs at 640×640 (Fig. 2). We are explicit about what we do and do not claim for it. It does not lead its class in clean accuracy and Section 4.3 reports that plainly; what we claim is its accuracy under vibration once the corruption

is in its training distribution, a combination no baseline we measured matches. The design choice that matters most for this is that it keeps a stride-4 level, since at the altitudes VisDrone covers a person occupies fewer than 32×32 pixels and detectors beginning at stride 8 discard that signal before detection begins.

Keeping stride 4 is what makes such a detector expensive. At 640×640 the P_2 map is 160×160, so every operation there costs $16\times$ what it costs at P_3, and in an earlier configuration of ours the P_2 head alone consumed 1.82 GMACs, 28% of total compute. Fig. 2 shows the three changes that cut the total to 4.6 GFLOPs without giving up stride 4: one Universal Inverted Bottleneck [15] block at P_2 instead of three, windowed self-attention [13] at P_5 only, and a depthwise-separable head in place of a five-convolution decoupled one. Deeper stages use cross-stage partial blocks [22].

Regression blends the Normalized Wasserstein Distance [23] with CIoU [25] by ground-truth area, since IoU gradients are unstable for tiny boxes; classification uses [11] and assignment follows [5], modified so each small ground truth gets several positives. Optimisation blends Muon [10] and SGD.

3.2 The VisDrone-Shake Protocol

Measuring vibration robustness needs a corruption model reflecting rotorcraft physics rather than a generic suite. Three effects dominate. **Directional motion blur**: camera displacement during exposure integrates the scene along the motion direction, modelled as convolution with a linear kernel. **Sensor noise**: blur reduces effective signal and automatic exposure compensates by raising gain, amplifying read noise. **Exposure oscillation**: rotor frequency beating against the rolling shutter gives frame-to-frame brightness fluctuation, applied multiplicatively.

The three are composed at three severity levels above clean, parameterised by (blur kernel length in pixels, noise σ, brightness range) as sev1 = (5, 0.02, $\pm$10%), sev2 = (9, 0.04, $\pm$20%) and sev3 = (15, 0.06, $\pm$30%), with blur orientation drawn uniformly per batch. Levels sev1 and sev2 correspond to a well-balanced airframe in steady flight; sev3 to severe excitation such as an unbalanced rotor. None of the three translates objects, so ground-truth boxes stay valid and the same operator serves as both a training perturbation and a fixed evaluation protocol.

Following [9] we report the mean corrupted precision mCP over the three corrupted levels and the retention ratio $\text{retain}\% = 100 \cdot \text{mCP}/\text{AP}_{\text{clean}}$, which measures how much of a detector's accuracy survives flight rather than how accurate it is on the ground. Two detectors with equal clean AP can differ widely in it.

The protocol is stochastic, so it must be seeded. Blur orientation, noise and brightness are drawn at random, so a single pass is one draw rather than a measurement: unseeded, retention for one checkpoint varied by more than a point between passes, enough to manufacture a modest improvement. We reseed

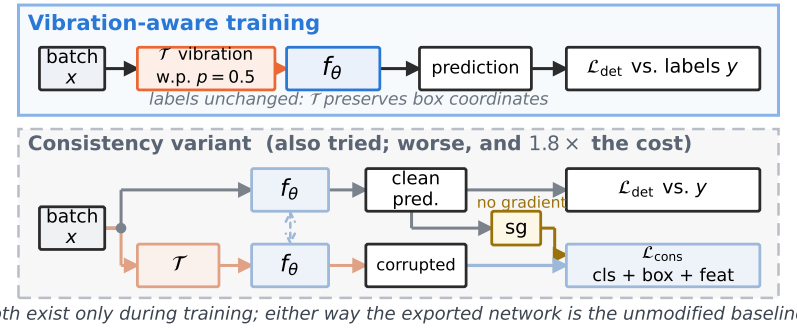


Fig. 3. Vibration-aware training (top) and the consistency variant we also tried (bottom). The method we recommend is the simple one: corrupt a fraction of each batch with $\mathcal{T}$ and compute the ordinary detection loss on the corrupted image. The variant forwards the batch twice and adds a consistency term against a stop-gradient copy of the clean prediction. Neither survives into deployment: the exported network is unchanged in either case.

before every severity pass and report mean and standard deviation over three seeds. Gains smaller than a benchmark's own run-to-run spread are not gains.

3.3 Vibration-Aware Training

Let $\mathcal{T}$ denote the stochastic vibration operator of Section 3.2, with severity and blur orientation resampled per batch. Fig. 3 contrasts the method with the variant of Section 4.2; the method is one line. During training, each batch x is replaced by $\mathcal{T}(x)$ with probability $p = 0.5$, and the ordinary detection loss is computed on whatever image the network receives:

$$\mathcal{L} = \mathcal{L}_{\text{det}}\big(f_\theta(\hat{x}), y\big), \qquad \hat{x} = \begin{cases} \mathcal{T}(x) & \text{with probability } p, \\ x & \text{otherwise.} \end{cases} \tag{1}$$

Because $\mathcal{T}$ preserves object coordinates the labels y need no adjustment, which is what makes the substitution legitimate at all. No module is added, no second forward pass is required, and the exported graph is structurally identical to the baseline, so inference cost is unchanged. The only training cost is the corruption operator, a depthwise convolution and two elementwise operations.

Why this is not simply data augmentation as usually understood. Standard detection augmentation samples nuisances the training distribution already contains: flips, scale jitter, colour shifts. $\mathcal{T}$ samples one it does *not*, because VisDrone is collected from stable platforms. This is therefore not a regularisation trick but a correction of a mismatch between the data a detector is trained on and the conditions it will meet, which is why the severities in Section 3.2 are chosen to bracket rotorcraft flight rather than to maximise validation score.

A consistency variant, which we report because it is worse. Before running the control, we expected that a stronger signal could be extracted from the corrupted view than the detection loss provides. The variant forwards both x and $\mathcal{T}(x)$ through the same weights, computes the detection loss on the clean branch only, and adds a consistency term that pulls the corrupted prediction towards a stop-gradient copy of the clean one,

$$\mathcal{L} = \mathcal{L}_{\text{det}}\big(f_\theta(x), y\big) + \lambda_{\text{vib}}\, \mathcal{L}_{\text{cons}}\big(f_\theta(\tilde{x}),\ \text{sg}[f_\theta(x)]\big), \qquad \lambda_{\text{vib}} = 0.5, \tag{2}$$

where $\mathcal{L}_{\text{cons}}$ sums, over pyramid levels, a temperature-scaled cross-entropy on objectness, a smooth L_1 term on boxes gated by clean-branch objectness, and a squared error between neck features. Exact forms are in the released code. The stop-gradient is not optional: without it the objective is minimised by degrading the clean prediction towards the corrupted one, lowering the loss while destroying the detector. The reasoning was that the clean prediction says more than the hard label, and that blur destroys exactly that structure. Section 4.2 shows it does not survive measurement.

Training procedure. We apply the corruption as a finetuning stage rather than from scratch: starting from a converged baseline checkpoint, we finetune for 120 epochs at batch size 8 with gradient accumulation 12, MuSGD at learning rate 2×10^{-3}, and a three-epoch warm-up before the corruption is switched on. The method can therefore be applied to a detector that has already been trained, without repeating its schedule. Checkpoints are selected by clean validation AP, the same rule used for the baseline, so no checkpoint in this paper is chosen using the robustness protocol it is evaluated on.

4 Experiments

4.1 Setup

We evaluate on the person categories of VisDrone-DET [26], merging `pedestrian` and `people` into one class, which matches a following system that must track any human regardless of pose: 6471 training and 548 validation images.

One evaluation code path for every model. Each detector family ships its own harness with its own confidence threshold, detection cap and area convention, and differences of several AP points can be produced by the harness alone. We therefore evaluate *every* model in *every* table below with one script: images are letterboxed to 640×640 by the same loader, predictions are thresholded at confidence 0.01, reduced by NMS at IoU 0.5, capped at 300 detections per image, and scored by `pycocotools` on identically constructed ground truth. Baselines receive the same pre-processed tensors as ours, so no model sees a different image. Every corrupted evaluation is repeated with three seeds $(0, 1, 2)$, reseeding the corruption sampler before each severity pass, and reported as mean $\pm$ sample standard deviation; clean evaluation is deterministic.

Table 1. VisDrone person subset, 640 × 640, `pycocotools`. Clean accuracy is deterministic; corrupted columns are mean ± std over three seeds. Within each block the rows differ in exactly one variable and share the schedule, learning rate and augmentation pipeline. Reading the SteadyDet-N block downwards identifies where the robustness comes from: not from the extra epochs, and not from the consistency term. The YOLO11-n block, added as a test of generality, shows the regime having no effect at all outside our detector family.

	clean			under vibration ($AP_{50:95}$)			
Model	$AP_{50:95}$	AP_{50}	AP_{75}	sev1	sev2	sev3	retain%
SteadyDet-N (1.17 M, 4.6 GFLOPs)							
baseline	0.1892	0.5127	0.0811	0.0792 ±.0026	0.0123 ±.0013	0.0040 ±.0005	16.8 ±.7
+ 120 ep, no vibration	0.1823	0.4984	0.0762	0.0761 ±.0028	0.0125 ±.0005	0.0040 ±.0005	16.9 ±.7
+ consistency variant	0.1746	0.4837	0.0728	0.1098 ±.0010	0.0392 ±.0021	0.0081 ±.0011	30.0 ±.6
+ vibration-aware	0.1708	0.4731	0.0715	**0.1223 ±.0014**	**0.0518 ±.0019**	**0.0111 ±.0002**	**36.2 ±.7**
SteadyDet-W (1.00 M, 12.9 GFLOPs)							
baseline	0.2349	0.5800	0.1305	0.1260 ±.0023	0.0258 ±.0022	0.0046 ±.0026	22.2 ±.9
+ consistency variant	0.2207	0.5561	0.1202	0.1309 ±.0026	0.0580 ±.0007	0.0132 ±.0016	30.5 ±.6
+ vibration-aware	0.2219	0.5511	0.1279	**0.1583 ±.0013**	**0.0839 ±.0001**	**0.0200 ±.0012**	**39.4 ±.3**
YOLO11-n, trained from scratch on the same data (2.59 M, 6.4 GFLOPs)							
baseline	0.2588	0.5842	0.1839	0.1398 ±.0057	0.0248 ±.0022	0.0052 ±.0021	21.9 ±1.1
+ 120 ep, no vibration	0.2641	0.5897	0.1857	0.1410 ±.0036	0.0226 ±.0026	0.0062 ±.0030	21.4 ±.9
+ vibration-aware	0.2579	0.5835	0.1806	0.1384 ±.0037	0.0227 ±.0017	0.0055 ±.0026	21.5 ±.8

Training. Training uses only VisDrone, with no COCO person data, so comparison against prior work is not confounded by extra training data. Baselines start from COCO-pretrained weights and ours from scratch, which favours them, and each family uses its own recommended optimiser and schedule rather than a shared one, since a shared one destroys some of them: under default CNN optimiser selection RT-DETR diverges by epoch four. All models train on a single RTX 4060 Laptop GPU (8 GB), one run per configuration.

4.2 Main Results

Table 1 reports the primary results. On SteadyDet-N the baseline retains 16.8 ± 0.7% of its clean accuracy under vibration; training with the corruption raises this to 36.2 ± 0.7%, a factor of 2.15. At sev2, the moderate vibration of an airframe in forward flight, the baseline retains 6.5% of its clean accuracy and is effectively blind, while the vibration-aware model retains 30.3% and is 4.2× better in absolute AP, while clean AP falls by 9.7%. For an airborne system that exchange is favourable: a detector a tenth less accurate on the ground and four times more accurate with the rotors running is the more useful one. The two middle rows are what make the result interpretable, and each was run because a reader would otherwise be entitled to doubt the first.

The gain is not an artefact of the extra epochs. The regime is applied as a finetuning stage, so the vibration-aware model has seen 120 epochs that the baseline has not. Finetuning the baseline for the same 120 epochs with the corruption switched off leaves retention at $16.9 \pm 0.7\%$ against the baseline's $16.8 \pm 0.7\%$, a difference well inside one standard deviation. Continued training contributes nothing. Clean accuracy does drift down over those epochs (0.1892 to 0.1823), so roughly a third of the clean-AP cost of the vibration-aware model is the finetuning rather than the corruption.

The consistency term does not help; it hurts. We expected the opposite. The variant of Eq. 2 reaches $30.0 \pm 0.6\%$, 6.2 points below the simple substitution and roughly nine times the seed-to-seed spread of either measurement, so the ordering is not noise. It is worse in absolute terms at every severity, not only after normalisation, and costs $1.8\times$ the training time. Our reading is that the consistency term, computing the detection loss on the clean branch only, never asks the network to *detect* under blur; it asks only that its corrupted predictions resemble its clean ones, a weaker requirement that can be satisfied by making both less committed. The simple substitution puts the corrupted image directly against the labels. We offer this as an explanation rather than a demonstration.

Generality, and where it stops. A regime that works on exactly one architecture is hard to distinguish from a fortunate interaction with it. This one carries across both of our detectors and then stops, and the stop is worth as much detail as the success. The middle block of Table 1 repeats the experiment on SteadyDet-W, an earlier detector of ours with a full-width stride-4 branch and non-separable five-convolution heads, so that despite slightly *fewer* parameters (1.00 against 1.17 M) it costs $2.8\times$ the compute and is the stronger on clean data. Nothing was retuned. Retention rises from $22.2 \pm 0.9\%$ to $39.4 \pm 0.3\%$, a gain of 17.2 points against 19.3, and the consistency variant again falls short at $30.5 \pm 0.6\%$. Direction, magnitude and the ordering of the two variants all reproduce on a network of different width, depth and head design.

The lower block applies the identical regime to YOLO11-n, and nothing happens: $21.9 \pm 1.1\%$ becomes $21.5 \pm 0.8\%$, with the no-vibration control at $21.4 \pm 0.9\%$. The consistency variant on the COCO-pretrained checkpoint likewise moved $20.1 \pm 1.2\%$ to $19.2 \pm 1.3\%$. Logging the corrupted-batch substitution during training confirms the treatment was applied and not silently skipped. We tested three explanations and none survived.

Pretraining. If the baseline already holds blur-tolerant features there is nothing left to teach. Retraining YOLO11-n from random initialisation leaves the regime at -0.3 points, exactly as from pretrained weights, and the pretrained checkpoint is the *less* robust of the two (20.1 against 21.9%).

Object scale. Our detectors carry a stride-4 level and YOLO11-n starts at stride 8, so perhaps only the finest level has anything to gain. Broken out on small objects the gain is +18.7 points against +19.3 overall, tracking the aggregate rather than concentrating in it, and nearly every instance in this subset is small

Table 2. VisDrone person subset, every row measured by the same evaluation code (Section 4.1). mCP is mean $AP_{50:95}$ over the three vibration levels, that is, accuracy *in flight*; retention is mCP/AP_{clean}, mean ± std over three seeds. YOLO and RT-DETR rows start from COCO-pretrained weights while SteadyDet rows train from scratch, which favours the baselines. *The clean and the mCP columns rank the table differently, and mCP is the one the aircraft experiences.*

Model	Params (M)	GFLOPs	AP_{50}	$AP_{50:95}$	mCP	retain%
RT-DETR-l	32.81	108.0	0.5909	0.2502†	0.0641	25.6 ±1.1
YOLOv10-n	2.71	8.4	0.6043	0.2730	0.0538	19.7 ±1.4
YOLOv8-n	3.01	8.2	0.6033	0.2684	0.0522	19.5 ±.8
YOLO26-n	2.50	5.8	0.5898	0.2675	0.0507	19.0 ±1.0
YOLO11-n	2.59	6.4	**0.6119**	**0.2768**	0.0557	20.1 ±1.2
+ vib.-aware	2.59	6.4	0.6057	0.2700	0.0542	20.1 ±1.3
SteadyDet-W	1.00	12.9	0.5800	0.2349	0.0521	22.2 ±.9
+ vib.-aware	1.00	12.9	0.5511	0.2219	**0.0874**	**39.4 ±.3**
SteadyDet-N	**1.17**	**4.6**	0.5127	0.1892	0.0318	16.8 ±.7
+ consistency	1.17	4.6	0.4837	0.1746	0.0524	30.0 ±.6
+ vib.-aware	**1.17**	**4.6**	0.4731	0.1708	0.0617	36.2 ±.7

† RT-DETR's reference implementation scale-fills rather than letterboxes its input; under our common pipeline its clean AP is depressed. Native harness: 0.3340. mCP and retention are ratios within that same pipeline and are unaffected.

for both families (AP_s 0.2741 of AP 0.2768 for YOLO11-n). There is no scale split available to carry the argument.

Headroom. SteadyDet-W starts at $22.2 \pm 0.9\%$ and YOLO11-n from scratch at $21.9 \pm 1.1\%$, the same point within noise. One gains 17.2 points and the other none.

What remains is the architecture or the loss, which our detectors share and YOLO11-n does not: a stride-4 branch, a normalised-Wasserstein and CIoU box objective, quality focal classification, and direct rather than distributional box regression. One non-responding model cannot separate these, so we state the claim at the scope the evidence supports. Vibration-aware training is a large, reproducible gain for the detector family of Section 3.1 and does nothing on the one outside architecture we tried; locating the boundary is the most useful experiment this work leaves open.

4.3 Comparison with Lightweight Detectors

Table 2 compares five detectors spanning three paradigms against ours. Every row, including the corrupted columns, came from the single evaluation path of Section 4.1. We report it in full rather than selectively, and the two accuracy columns disagree about who wins.

On clean data our architecture does not lead its class. Every YOLO nano variant exceeds it, and we do not explain that away. COCO pretraining

is one factor and our from-scratch training a second, but the gap is larger than those plausibly account for, and efficiency does not close it: 4.6 GFLOPs against 5.8 for the closest baseline is a 21% saving that does not pay for the accuracy difference. On the benchmark, we lose.

Under vibration the ranking reverses, and that is the condition that matters. The mCP column orders the table almost opposite to the clean column. SteadyDet-N with vibration-aware training reaches 0.0617 against 0.0557 for the strongest baseline, at 45% of its parameters and 72% of its compute. RT-DETR-l leads it on the mCP average (0.0641), but not where it counts: at sev2, the severity of an airframe in ordinary forward flight, RT-DETR reaches 0.0389 ± 0.0014 against our 0.0518 ± 0.0019, a gap of five pooled standard deviations from a network with 28× the parameters and 23× the compute. At sev3 the two are within noise (0.0100 ± 0.0017 against 0.0111 ± 0.0002) and we claim no ordering there. The margin over YOLO11-n at sev2 is a factor of 2.5.

SteadyDet-W leads the whole table at 0.0874, on fewer parameters than SteadyDet-N but 2.8× the compute, and we say plainly that it is the more accurate of our two networks under vibration. SteadyDet-N is the primary model because compute, not parameter count, is what binds on the target platform: a following system must fit detection, tracking and control into one frame budget, and 12.9 GFLOPs does not leave room for the rest. Per unit of compute the ordering is unambiguous, with SteadyDet-N returning 0.0134 mCP per GFLOP against 0.0087 for the best baseline and 0.0068 for SteadyDet-W.

A reader should object that we compare our treated models against untreated baselines, and that objection is why Section 4.2 matters here. We applied the identical regime to YOLO11-n and it does not respond: treated, it reaches mCP 0.0542, marginally *below* its untreated 0.0557. The comparison is therefore between each detector at its own best, not against a handicapped one. We tested only YOLO11-n this way, so for the other three baselines the objection stands unanswered, and we say so rather than assume the null generalises.

The claim is accordingly narrow and, we think, the useful one: for a vibrating airframe with a few GFLOPs of budget, the detector to choose is not the one that wins the benchmark, but a smaller one whose training distribution contains the corruption. Our untreated baseline is the worst row in the table at 0.0318, so neither half of that sentence can be dropped.

Robustness is not a by-product of accuracy. Fig. 4 plots clean accuracy against retention. Hendrycks and Dietterich already reported, for ImageNet classifiers, that robustness measured relative to clean accuracy barely improved across a decade of architectural progress [9]; we find the same decoupling holds for compact aerial detectors under a corruption they were never designed for, which is worth stating because the detectors here span four years and three paradigms. Four YOLO generations sit within a point of one another at 19 to 20% despite their differences, and the transformer reaches $25.6 \pm 1.1\%$ only by

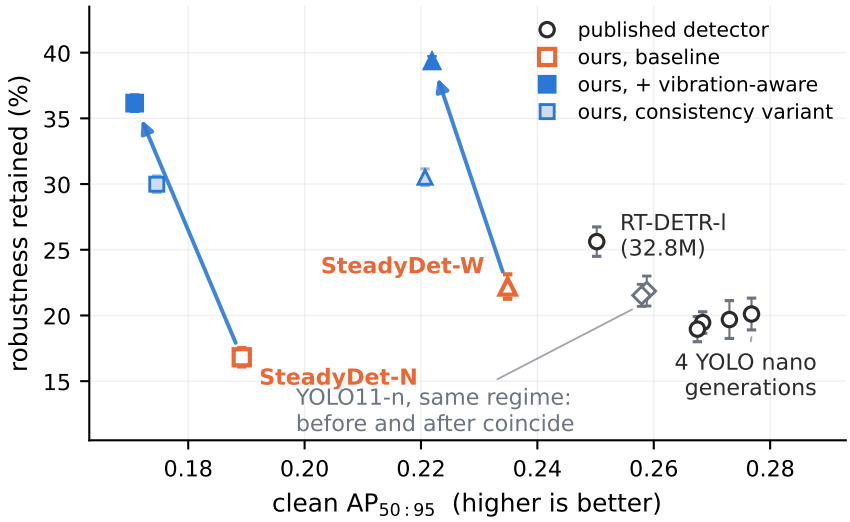


Fig. 4. Clean accuracy against robustness retained, one point per model, error bars over three seeds. The published detectors occupy the lower right: more accuracy, no more robustness, with four YOLO generations landing within a point of each other. The arrows are the effect of vibration-aware training; the pale markers are the consistency variant, which lands short of it. The two grey diamonds are YOLO11-n before and after the identical regime, and they sit on top of one another.

spending 28× our parameters. Robustness therefore has to be measured and optimised separately rather than expected to follow from clean accuracy. The arrows show the alternative: a large robustness gain for a small loss of clean accuracy, obtained by changing the training distribution rather than the network.

Our re-measured clean figures sit slightly below the detectors' own harnesses (0.2684 against 0.2787 for YOLOv8-n) because each applies different confidence, detection-count and area conventions, by margins comparable to the gaps between the baselines themselves. That is why harnesses cannot be mixed within one table.

5 Limitations

The corruption model is synthetic, and it is the load-bearing assumption. VisDrone-Shake is built from the known mechanisms of rotorcraft image degradation, but its parameters bracket flight rather than being calibrated against accelerometer spectra measured on our own airframe. Every result in this paper is therefore a statement about a *model* of vibration, and we make no claim about the magnitude of the effect on any particular aircraft. Only measurement can settle that: flying the same detector with a synchronised IMU log, fitting $\mathcal{T}$ to the accelerometer spectrum, and re-evaluating. That is the single

most valuable extension of this work, and until it is done the ordering of the models under vibration is better trusted than the absolute numbers.

One dataset, one corruption family. Every number here comes from the person subset of a single benchmark. VisDrone is the standard choice for this task and its instance statistics are the ones our detector targets, but a corruption robustness result measured on one dataset is a statement about that dataset until it is repeated. We also model one corruption family, the one rotors produce; nothing here speaks to rain, fog, low light or motion of the subject rather than the camera, all of which a fielded UAV also meets.

The scope of the regime is bounded on one side only. The gain reproduces on both of our detectors and is absent on the one outside architecture we tried. Section 4.2 rejects pretraining, object scale and remaining headroom as explanations, which narrows the cause to the architecture or the loss without identifying it, and one non-responding model cannot. We have therefore demonstrated a large effect within one detector family and one negative instance outside it, with the boundary unknown. A practitioner should treat the regime as worth trying rather than as established for their detector, and should run the two controls of Table 1 when they do.

Cost, tuning and statistical power. Why the consistency term underperforms is measured but not explained: we offer the reading that the detection loss never reaches the corrupted image, without testing it, so take the ordering as measured and the reason as conjecture. Clean accuracy falls by 9.7%, roughly a third of that from the finetuning itself, and we did not try to recover the rest by annealing the corruption probability late in training. We report no sweep over p or the severity distribution, so those values are reasonable rather than tuned. Each configuration is trained once, so our error bars capture evaluation but not training variance. Frames are processed independently, so the temporal redundancy vibration itself creates, successive frames blurring along different directions, is available only to the downstream tracker.

6 Conclusion

SteadyDet-N, at 1.17 M parameters and 4.6 GFLOPs, is the least accurate detector in our comparison on clean VisDrone and, within its compute budget, the most accurate one under vibration. Getting there required both halves of this paper. The failure it starts from is general: under a single evaluation protocol every detector we tested, from 1.17 to 32.8 M parameters, loses most of its accuracy to the vibration intrinsic to rotor-driven flight, so the problem belongs to the regime rather than to any one architecture. The fix is cheaper than the problem. Putting a physically motivated vibration model into the training distribution raises retention from $16.8 \pm 0.7\%$ to $36.2 \pm 0.7\%$ while leaving the deployed network untouched, and a matched control confirms the extra epochs contribute nothing. Neither the network nor the regime does this alone: untreated, the net-

work is the worst row in our table, and the same regime on YOLO11-n moves nothing.

We also report two results we did not want: the consistency formulation we expected to extract more from the corrupted view reaches only $30.0 \pm 0.6\%$ at $1.8\times$ the training cost, and neither formulation moves YOLO11-n. We think this is the more useful half of the paper. The default when a robustness intervention works is to credit the mechanism that was designed; here the control says the gain came from the data and the designed mechanism was in the way, and the generality test says we do not yet know what the gain depends on. Both are conclusions a reader can act on, and neither would have appeared without running the controls.

One lesson extends beyond our detector: a stochastic corruption benchmark must be seeded and reported with error bars, since run-to-run spread on a single checkpoint was large enough here to manufacture a modest improvement. The extensions we consider most valuable are calibrating the corruption against measured airframe spectra, and identifying which property of a detector decides whether the regime helps it at all.

Acknowledgements. **[Funding, laboratory, advisor.]**

References

1. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv:1503.02531 (2015)
2. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. In: ICML (2020)
3. Chen, C., et al.: RemDet: rethinking efficient model design for UAV object detection. In: AAAI (2025)
4. Cheng, G., Yuan, X., Yao, X., Yan, K., Zeng, Q., Xie, X., Han, J.: Towards large-scale small object detection: survey and benchmarks. IEEE TPAMI (2023)
5. Feng, C., Zhong, Y., Gao, Y., Scott, M.R., Huang, W.: TOOD: task-aligned one-stage object detection. In: ICCV (2021)
6. French, G., Mackiewicz, M., Fisher, M.: Self-ensembling for visual domain adaptation. In: ICLR (2018)
7. Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: exceeding YOLO series in 2021. arXiv:2107.08430 (2021)
8. Grill, J.B., et al.: Bootstrap your own latent: a new approach to self-supervised learning. In: NeurIPS (2020)
9. Hendrycks, D., Dietterich, T.: Benchmarking neural network robustness to common corruptions and perturbations. In: ICLR (2019)
10. Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., Bernstein, J.: Muon: an optimizer for hidden layers in neural networks (2024)
11. Li, X., Wang, W., Wu, L., Chen, S., Hu, X., Li, J., Tang, J., Yang, J.: Generalized focal loss: learning qualified and distributed bounding boxes for dense object detection. In: NeurIPS (2020)
12. Michaelis, C., et al.: Benchmarking robustness in object detection: autonomous driving when winter is coming. arXiv:1907.07484 (2019)

13. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin transformer: hierarchical vision transformer using shifted windows. In: ICCV (2021)
14. Nah, S., Kim, T.H., Lee, K.M.: Deep multi-scale convolutional neural network for dynamic scene deblurring. In: CVPR (2017)
15. Qin, D., et al.: MobileNetV4: universal models for the mobile ecosystem. In: ECCV (2024)
16. Sajjadi, M., Javanmardi, M., Tasdizen, T.: Regularization with stochastic transformations and perturbations for deep semi-supervised learning. In: NeurIPS (2016)
17. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: MobileNetV2: inverted residuals and linear bottlenecks. In: CVPR (2018)
18. Sohn, K., et al.: FixMatch: simplifying semi-supervised learning with consistency and confidence. In: NeurIPS (2020)
19. Tan, M., Pang, R., Le, Q.V.: EfficientDet: scalable and efficient object detection. In: CVPR (2020)
20. Tarvainen, A., Valpola, H.: Mean teachers are better role models: weight-averaged consistency targets improve semi-supervised deep learning results. In: NeurIPS (2017)
21. Wang, A., et al.: YOLOv10: real-time end-to-end object detection. In: NeurIPS (2024)
22. Wang, C.Y., Liao, H.Y.M., Wu, Y.H., Chen, P.Y., Hsieh, J.W., Yeh, I.H.: CSPNet: a new backbone that can enhance learning capability of CNN. In: CVPR Workshops (2020)
23. Wang, J., Xu, C., Yang, W., Yu, L.: A normalized Gaussian Wasserstein distance for tiny object detection. arXiv:2110.13389 (2021)
24. Yu, X., Gong, Y., Jiang, N., Ye, Q., Han, Z.: Scale match for tiny person detection. In: WACV (2020)
25. Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., Ren, D.: Distance-IoU loss: faster and better learning for bounding box regression. In: AAAI (2020)
26. Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and tracking meet drones challenge. IEEE TPAMI **44**(11), 7380-7399 (2021)